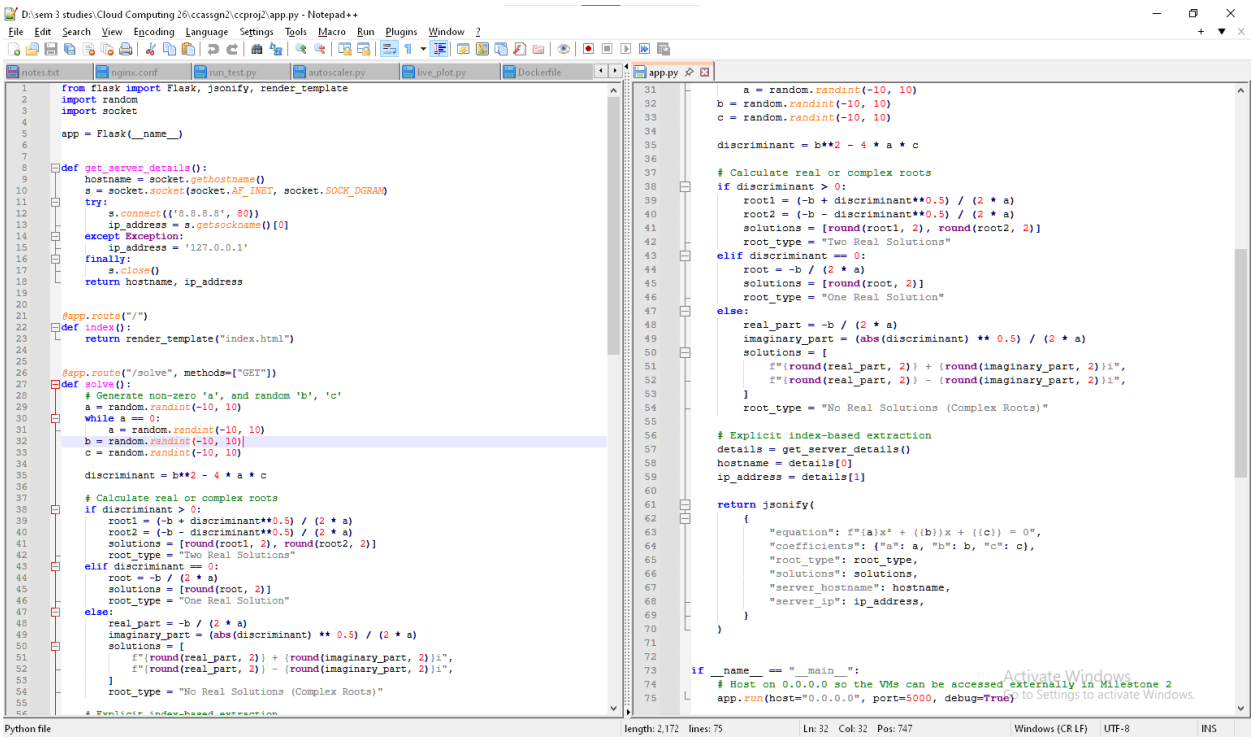
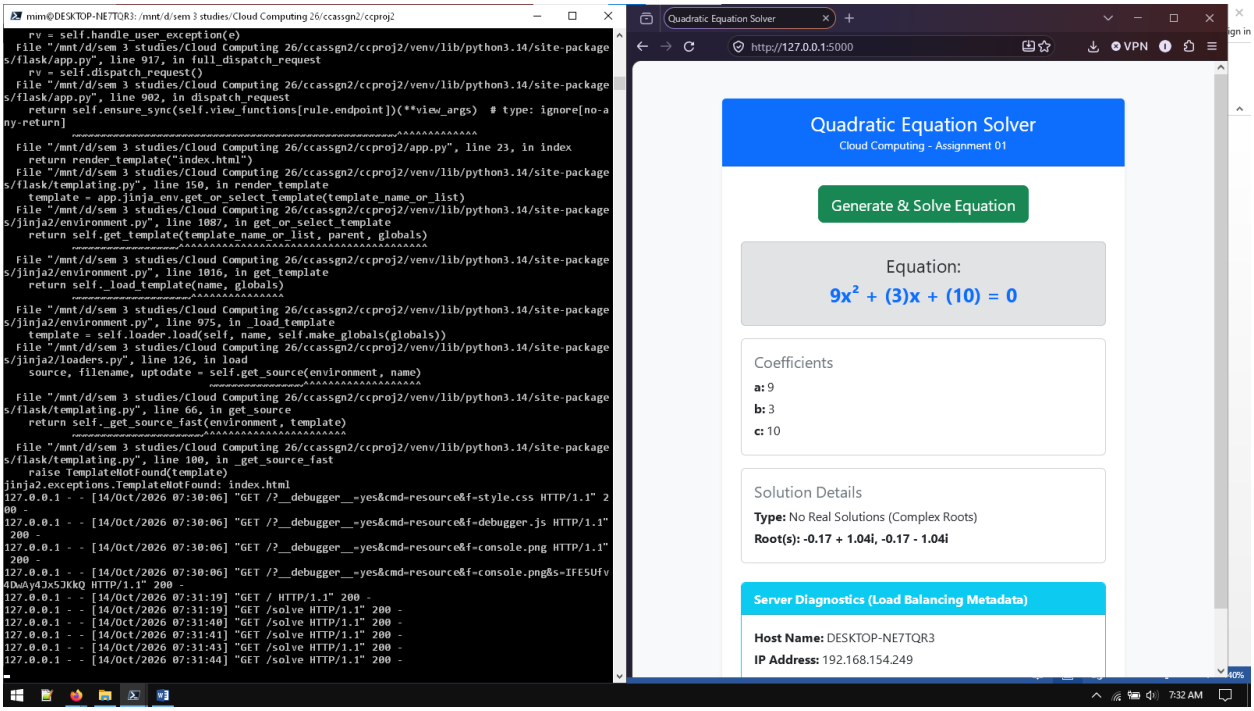
Milestone 1: Snippet of App running and Code on Host OS.





Milestone 02: Host your Web Application in a Docker Container

1. Docker file

s.txt | nginx.conf | run_test.py | autoscaler.py | live_plot.py | Dockerfile

```
# Use an official lightweight Python runtime
FROM python:3.11-slim

# Set work directory
WORKDIR /app

# Prevent Python from writing .pyc files and enable unbuffered logging
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

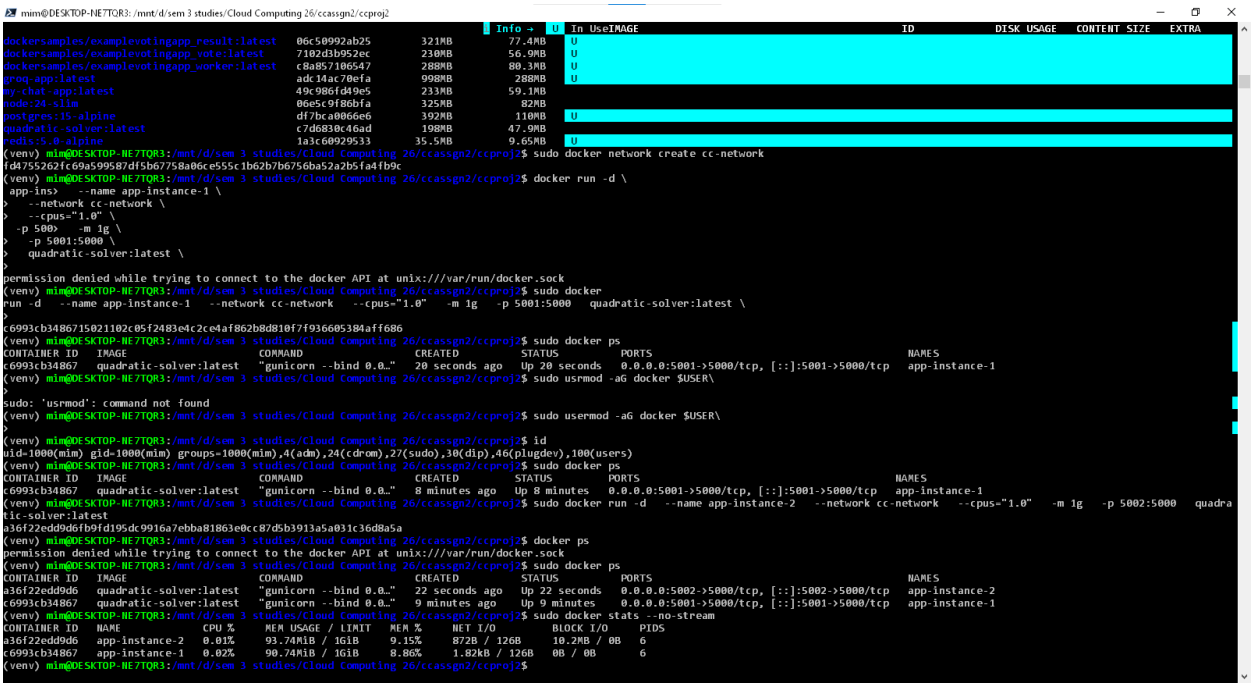
# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application files and templates
COPY app.py .
COPY templates/ ./templates/

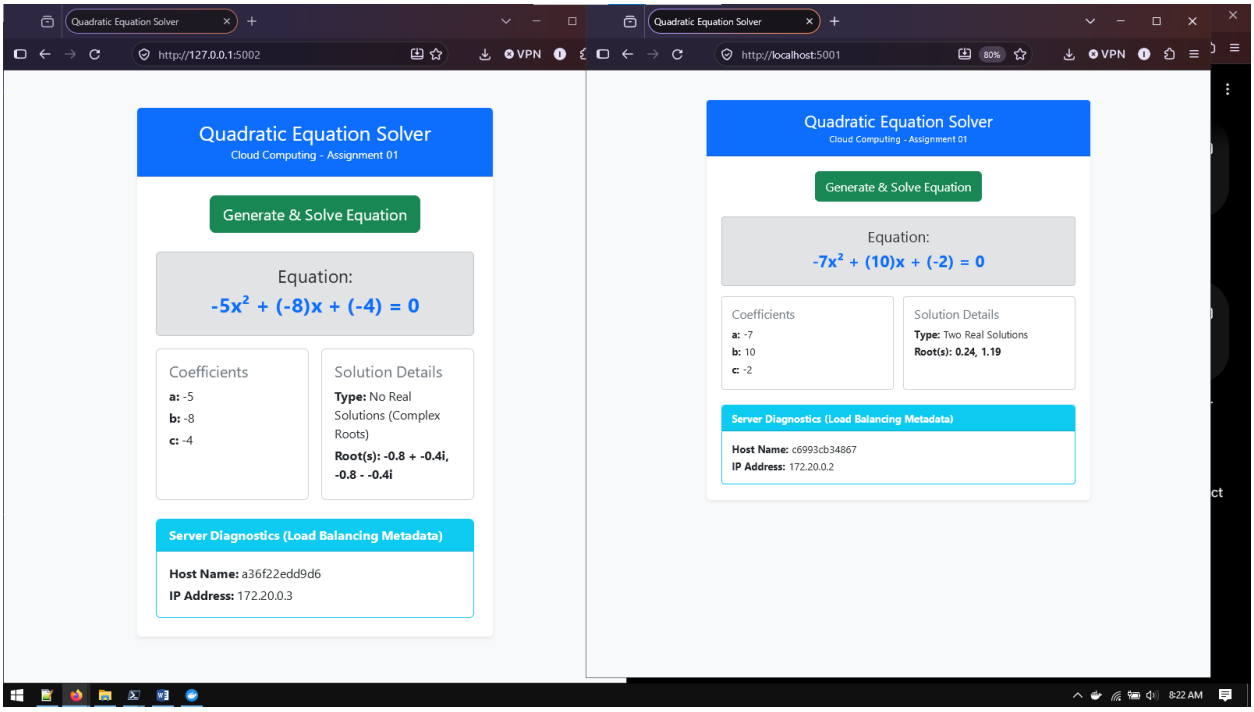
# Expose Flask / Gunicorn port
EXPOSE 5000

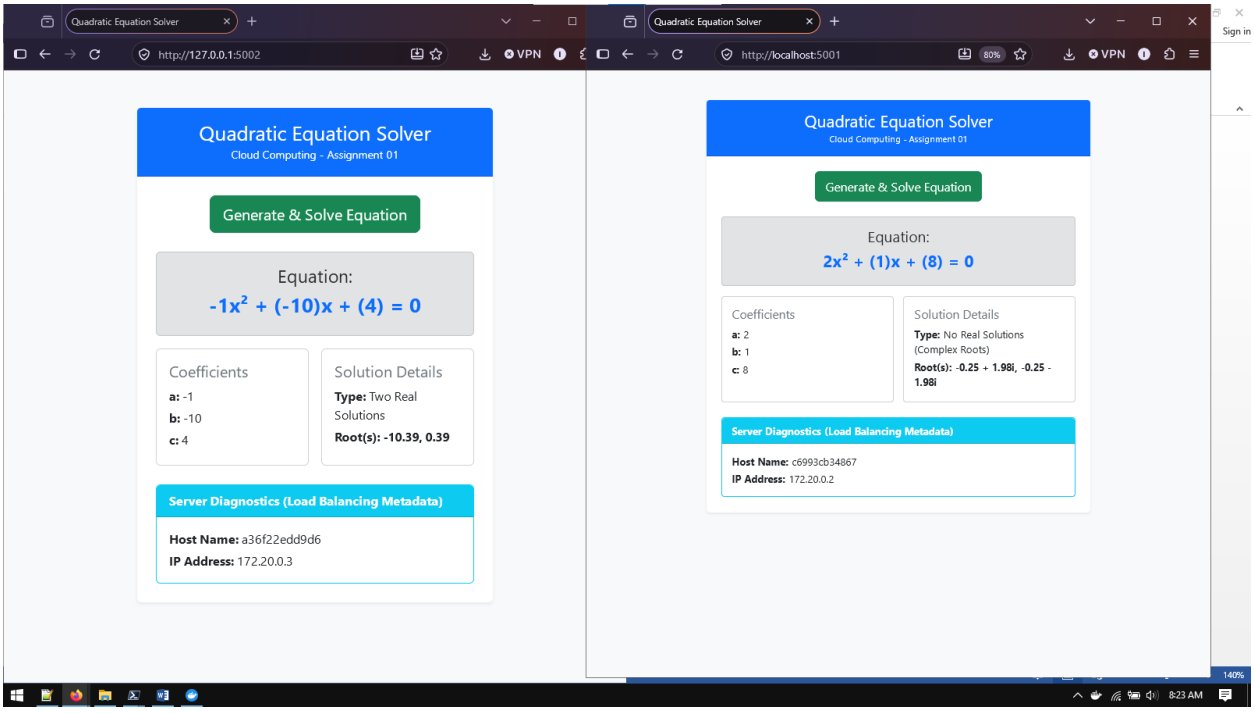
# Run the app with Gunicorn
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "4", "app:app"]
```

2. Container app running on local host port 5001 and 5002 and container stats



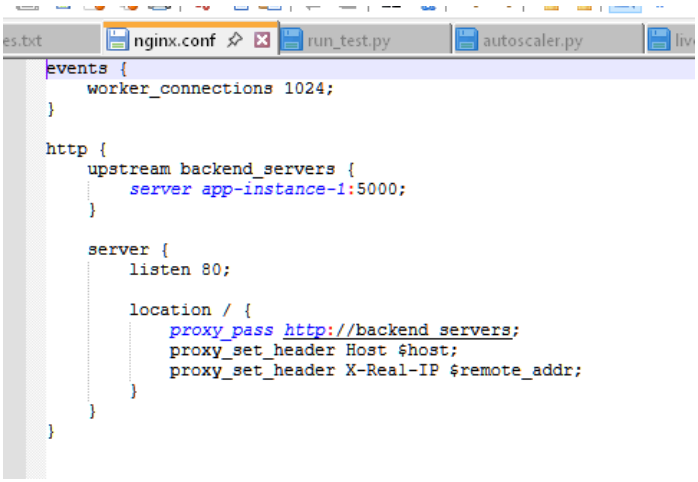
3. Both container's Web App running on HOST OS



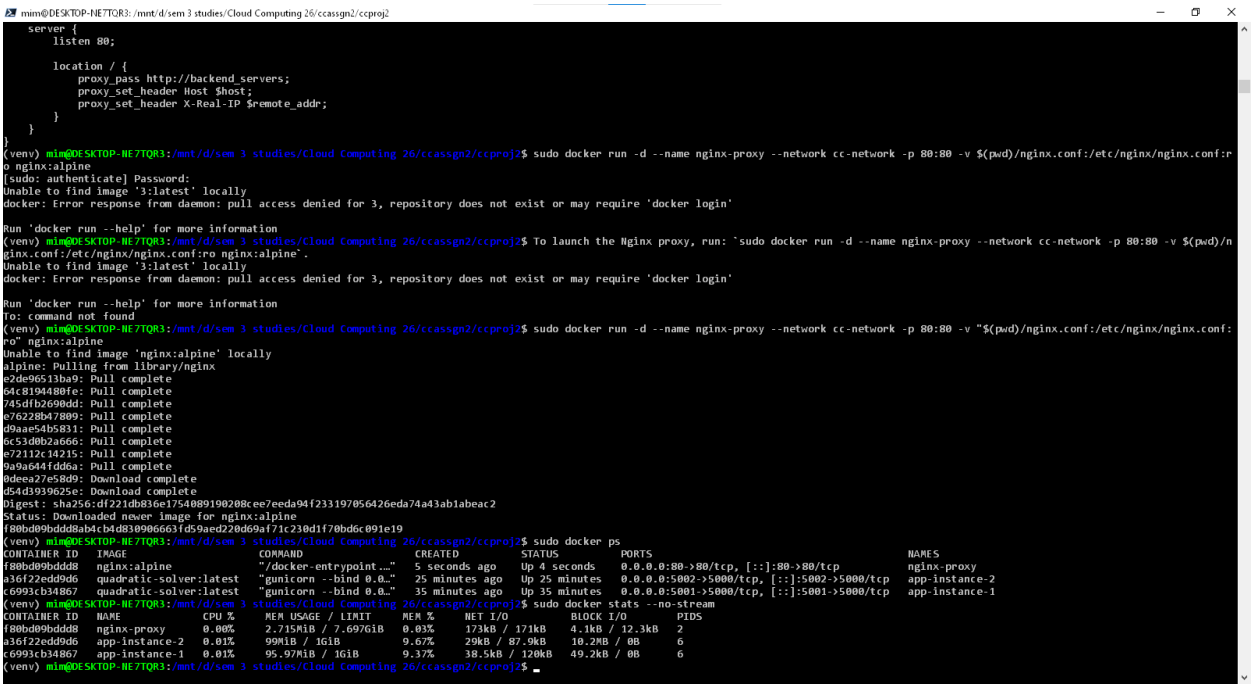


Milestone 03: Deploy Nginx as a Proxy Server in a Docker Container

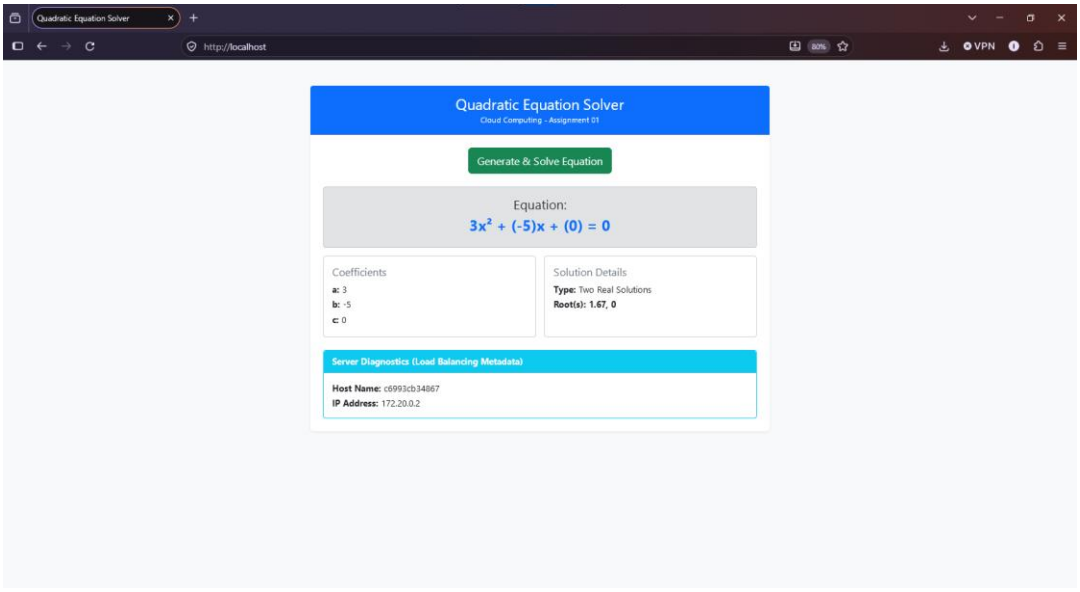
1. NGINX Configurations

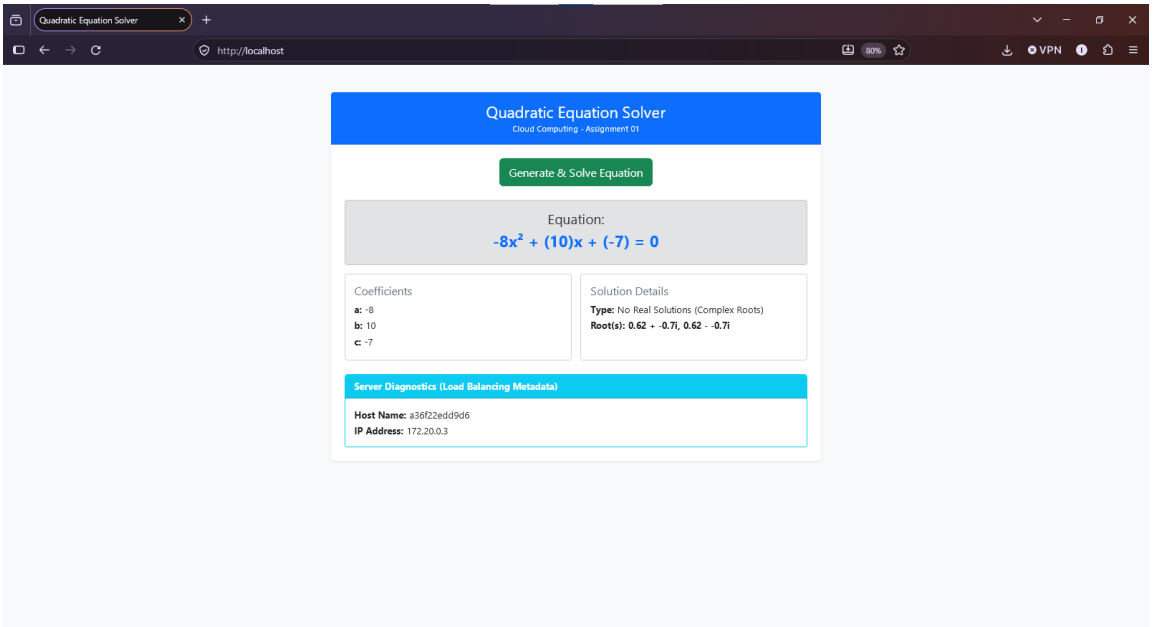


2. Creating NGINX container and initializing



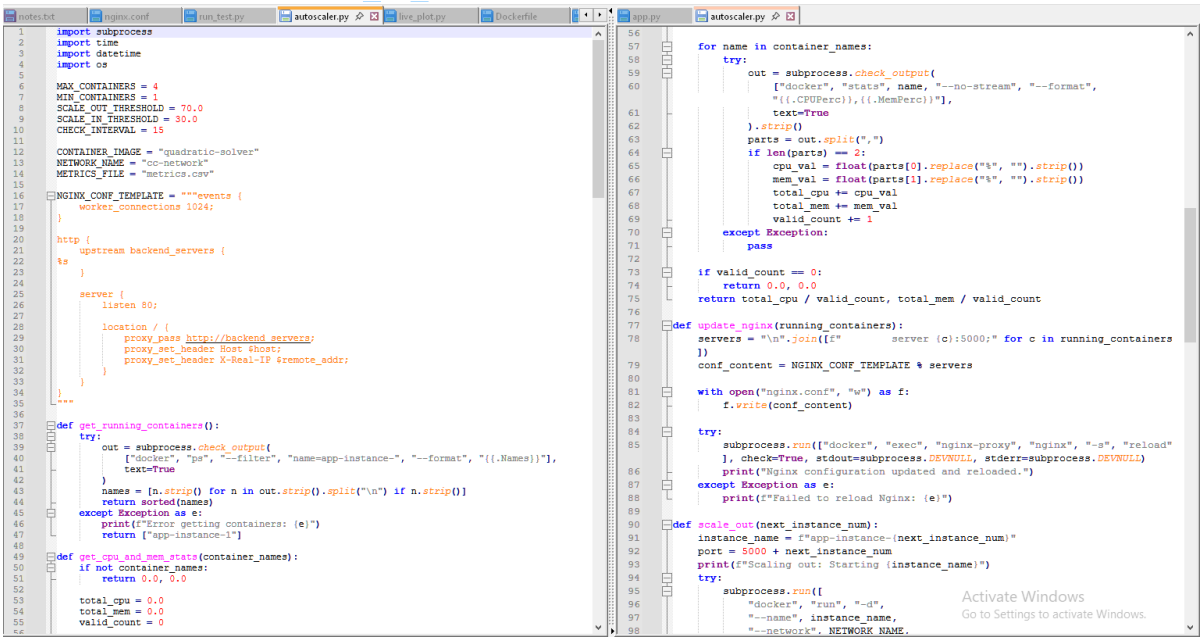
3. NGINX is Load Balancing among 2 load instances





Milestone 04: Metrics Collection and Auto scaling

1. Auto scalar script



```python
import subprocess
import time
import datetime
import os

MAX_CONTAINERS = 4
MIN_CONTAINERS = 1
SCALE_OUT_THRESHOLD = 70.0
SCALE_IN_THRESHOLD = 30.0
CHECK_INTERVAL = 15

CONTAINER_IMAGE = "quadratic-solver"
NETWORK_NAME = "cc-network"
METRICS_FILE = "metrics.csv"

NGINX_CONF_TEMPLATE = """events {
    worker_connections 1024;
}

http {
    upstream backend_servers {
%s
    }

    server {
        listen 80;

        location / {
            proxy_pass http://backend_servers;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
"""

def get_running_containers():
    try:
        out = subprocess.check_output(
            ["docker", "ps", "--filter", "name=app-instance-", "--format", "{{.Names}}"],
            text=True
        )
        names = [n.strip() for n in out.strip().split("\n") if n.strip()]
        return sorted(names)
    except Exception as e:
        print(f"Error getting containers: {e}")
        return ["app-instance-1"]

def get_cpu_and_mem_stats(container_names):
    if not container_names:
        return 0.0, 0.0

    total_cpu = 0.0
    total_mem = 0.0
    valid_count = 0
```

```python
    for name in container_names:
        try:
            out = subprocess.check_output(
                ["docker", "stats", name, "--no-stream", "--format",
                "{{.CPUPerc}},{{.MemPerc}}"],
                text=True
            ).strip()
            parts = out.split(",")
            if len(parts) == 2:
                cpu_val = float(parts[0].replace("%", "").strip())
                mem_val = float(parts[1].replace("%", "").strip())
                total_cpu += cpu_val
                total_mem += mem_val
                valid_count += 1
        except Exception:
            pass

    if valid_count == 0:
        return 0.0, 0.0
    return total_cpu / valid_count, total_mem / valid_count

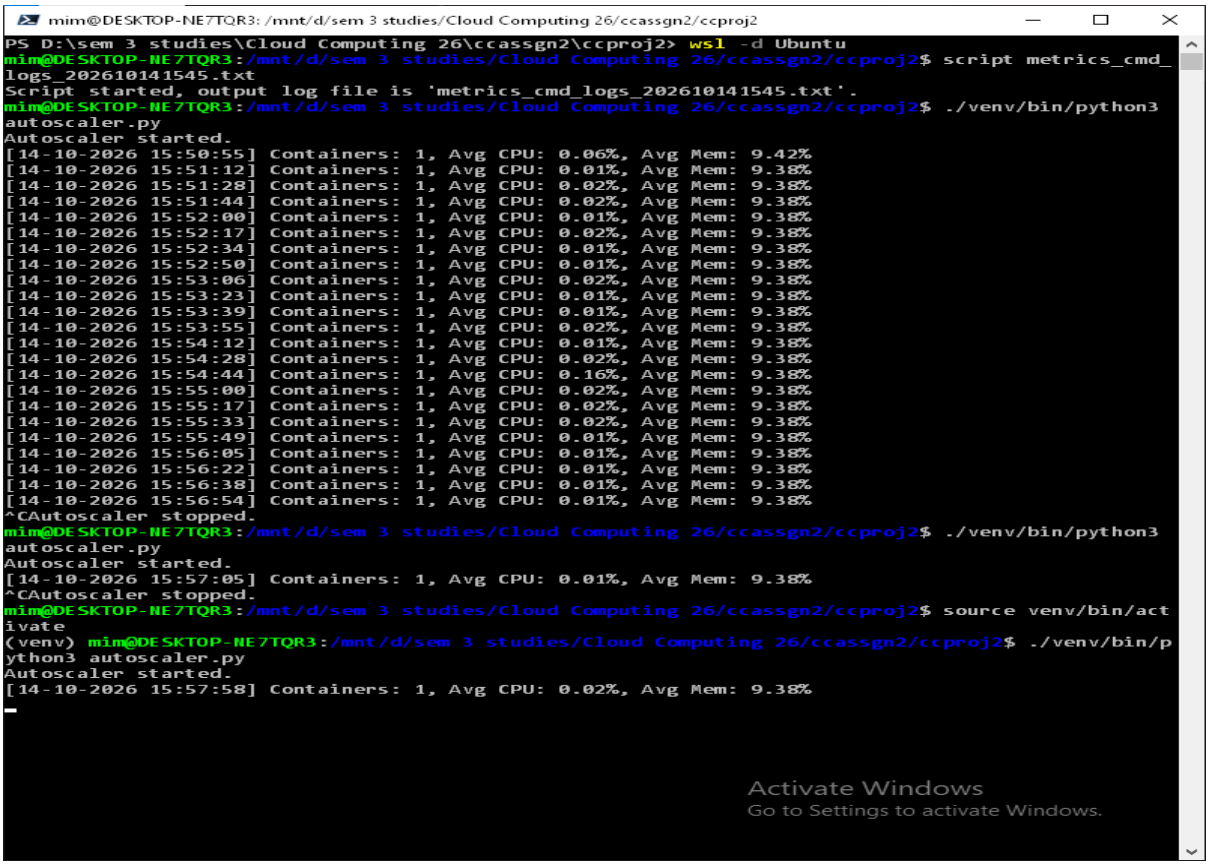
def update_nginx(running_containers):
    servers = "\n".join([f"        server {c}:5000;" for c in running_containers
    ])
    conf_content = NGINX_CONF_TEMPLATE % servers

    with open("nginx.conf", "w") as f:
        f.write(conf_content)

    try:
        subprocess.run(["docker", "exec", "nginx-proxy", "nginx", "-s", "reload"
        ], check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
        print("Nginx configuration updated and reloaded.")
    except Exception as e:
        print(f"Failed to reload Nginx: {e}")

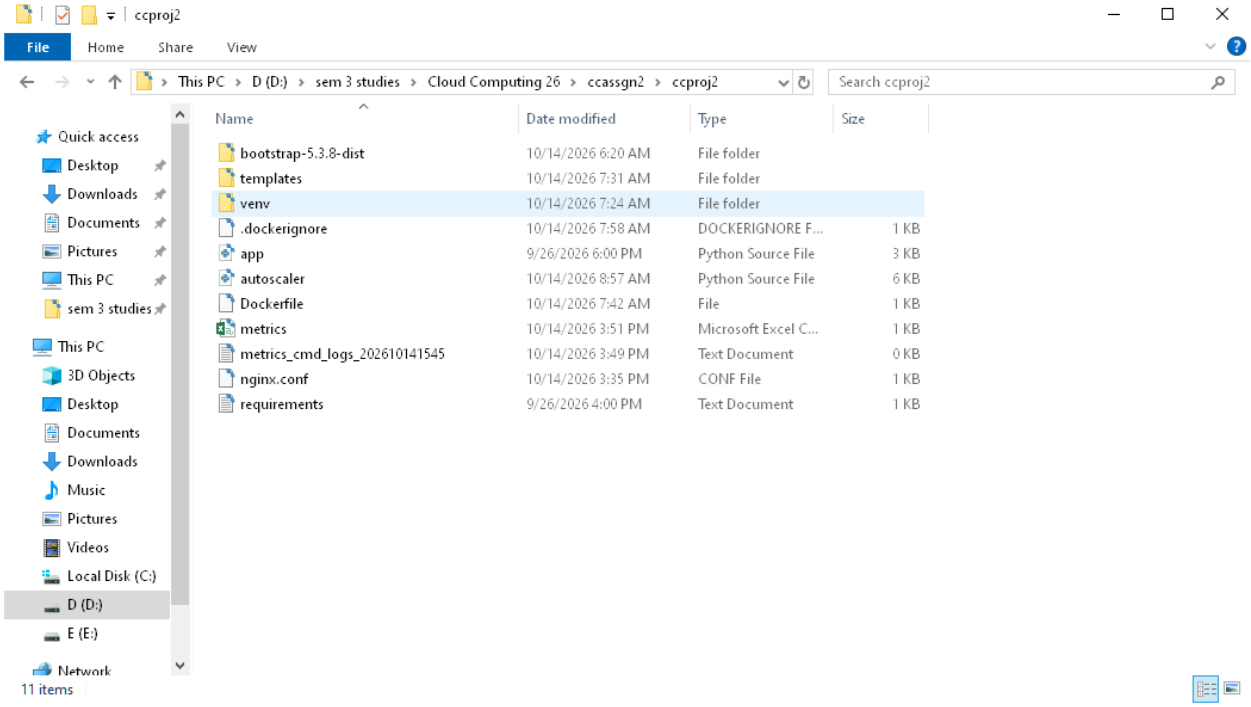
def scale_out(next_instance_num):
    instance_name = f"app-instance-{next_instance_num}"
    port = 5000 + next_instance_num
    print(f"Scaling out: Starting {instance_name}")
    try:
        subprocess.run([
            "docker", "run", "-d",
            "--name", instance_name,
            "--network", NETWORK_NAME,
```

2. Auto Scalar script has been activated and logged



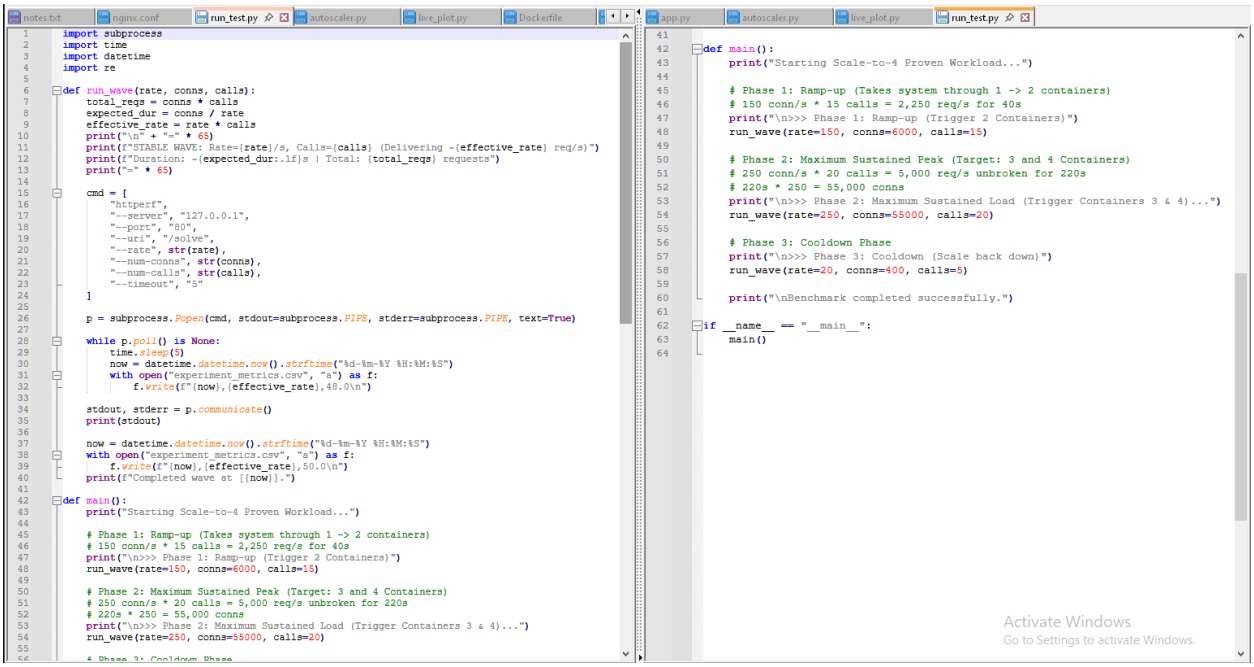
```
mim@DESKTOP-NE7TQR3: /mnt/d/sem 3 studies/Cloud Computing 26/ccassgn2/ccproj2
PS D:\sem 3 studies\Cloud Computing 26\ccassgn2\ccproj2> wsl -d Ubuntu
mim@DESKTOP-NE7TQR3:/mnt/d/sem 3 studies/Cloud Computing 26/ccassgn2/ccproj2$ script metrics_cmd_
logs_202610141545.txt
Script started, output log file is 'metrics_cmd_logs_202610141545.txt'.
mim@DESKTOP-NE7TQR3:/mnt/d/sem 3 studies/Cloud Computing 26/ccassgn2/ccproj2$ ./venv/bin/python3
autoscaler.py
Autoscaler started.
[14-10-2026 15:50:55] Containers: 1, Avg CPU: 0.06%, Avg Mem: 9.42%
[14-10-2026 15:51:12] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:51:28] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:51:44] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:52:00] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:52:17] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:52:34] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:52:50] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:53:06] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:53:23] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:53:39] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:53:55] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:54:12] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:54:28] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:54:44] Containers: 1, Avg CPU: 0.16%, Avg Mem: 9.38%
[14-10-2026 15:55:00] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:55:17] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:55:33] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
[14-10-2026 15:55:49] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:56:05] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:56:22] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:56:38] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
[14-10-2026 15:56:54] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
^CAutoscaler stopped.
mim@DESKTOP-NE7TQR3:/mnt/d/sem 3 studies/Cloud Computing 26/ccassgn2/ccproj2$ ./venv/bin/python3
autoscaler.py
Autoscaler started.
[14-10-2026 15:57:05] Containers: 1, Avg CPU: 0.01%, Avg Mem: 9.38%
^CAutoscaler stopped.
mim@DESKTOP-NE7TQR3:/mnt/d/sem 3 studies/Cloud Computing 26/ccassgn2/ccproj2$ source venv/bin/act
ivate
(venv) mim@DESKTOP-NE7TQR3:/mnt/d/sem 3 studies/Cloud Computing 26/ccassgn2/ccproj2$ ./venv/bin/p
ython3 autoscaler.py
Autoscaler started.
[14-10-2026 15:57:58] Containers: 1, Avg CPU: 0.02%, Avg Mem: 9.38%
```

3. Project Folder



Milestone 05: Synthetic Workload Generation and Auto scaling Evaluation

1. Load Generation Script



```python
import subprocess
import time
import datetime
import re

def run_wave(rate, conns, calls):
    total_reqs = conns * calls
    expected_dur = conns / rate
    effective_rate = rate * calls
    print("\n" + "=" * 65)
    print(f"STABLE WAVE: Rate={rate}/s, Calls={calls} (Delivering ~{effective_rate} req/s)")
    print(f"Duration: ~{expected_dur:.1f}s | Total: {total_reqs} requests")
    print("=" * 65)

    cmd = [
        "httperf",
        "--server", "127.0.0.1",
        "--port", "80",
        "--uri", "/solve",
        "--rate", str(rate),
        "--num-conns", str(conns),
        "--num-calls", str(calls),
        "--timeout", "5"
    ]

    p = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True)

    while p.poll() is None:
        time.sleep(5)
        now = datetime.datetime.now().strftime("%d-%m-%Y %H:%M:%S")
        with open("experiment_metrics.csv", "a") as f:
            f.write(f"{now},{effective_rate},48.0\n")

    stdout, stderr = p.communicate()
    print(stdout)

    now = datetime.datetime.now().strftime("%d-%m-%Y %H:%M:%S")
    with open("experiment_metrics.csv", "a") as f:
        f.write(f"{now},{effective_rate},50.0\n")
    print(f"Completed wave at [{now}].")

def main():
    print("Starting Scale-to-4 Proven Workload...")

    # Phase 1: Ramp-up (Takes system through 1 -> 2 containers)
    # 150 conn/s * 15 calls = 2,250 req/s for 40s
    print("\n>>> Phase 1: Ramp-up (Trigger 2 Containers)")
    run_wave(rate=150, conns=6000, calls=15)

    # Phase 2: Maximum Sustained Peak (Target: 3 and 4 Containers)
    # 250 conn/s * 20 calls = 5,000 req/s unbroken for 220s
    # 220s * 250 = 55,000 conns
    print("\n>>> Phase 2: Maximum Sustained Load (Trigger Containers 3 & 4)...")
    run_wave(rate=250, conns=55000, calls=20)

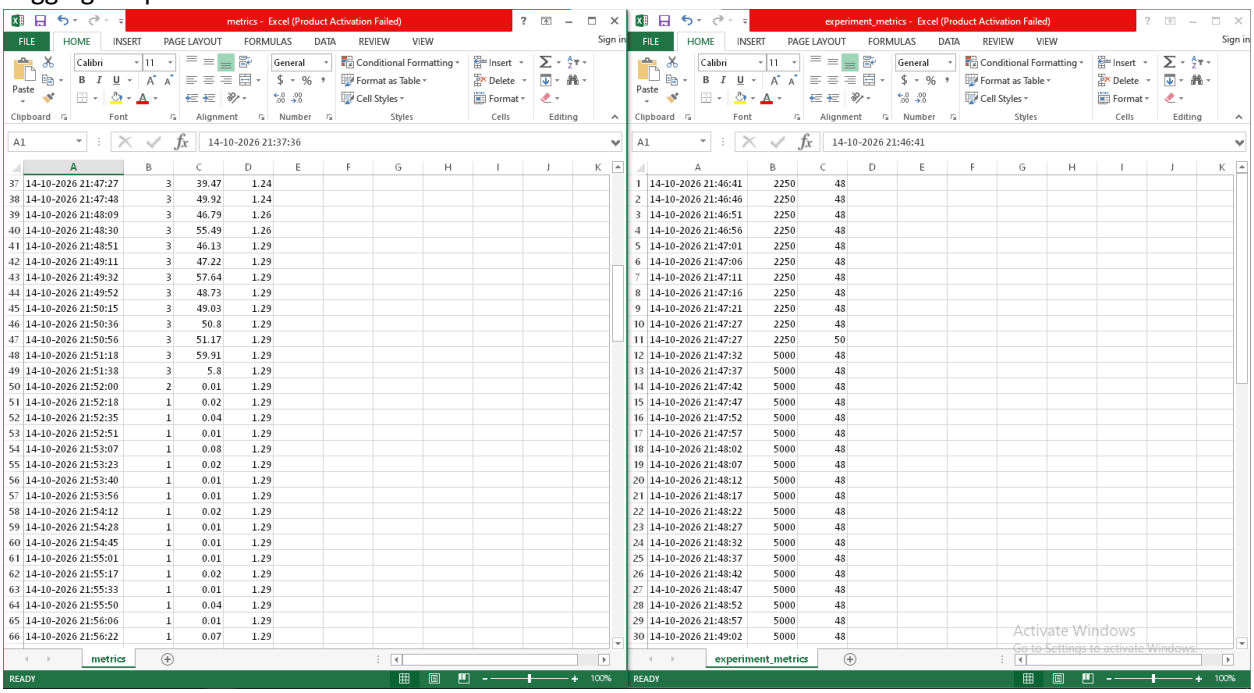
    # Phase 3: Cooldown Phase
    print("\n>>> Phase 3: Cooldown (Scale back down)")
    run_wave(rate=20, conns=400, calls=5)

    print("\nBenchmark completed successfully.")

if __name__ == "__main__":
    main()
```

Activate Windows
Go to Settings to activate Windows.

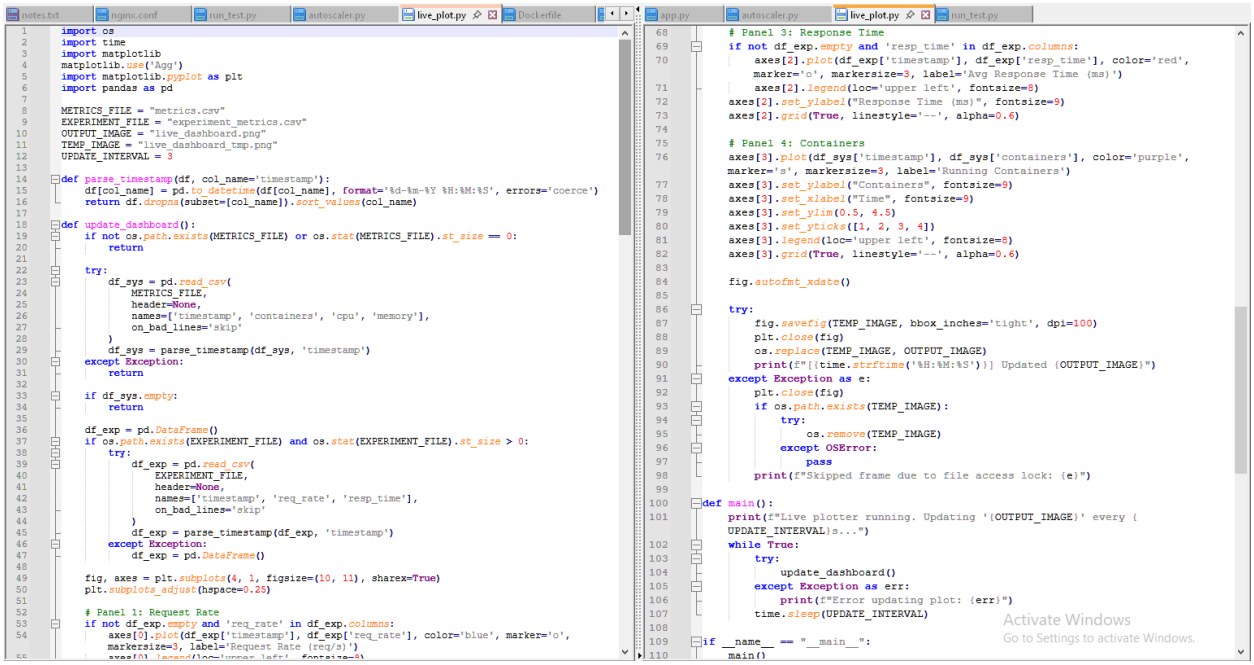
2. Logging Graph value in CSV File



	A	B	C	D
37	14-10-2026 21:47:27	3	39.47	1.24
38	14-10-2026 21:47:48	3	49.92	1.24
39	14-10-2026 21:48:09	3	46.79	1.26
40	14-10-2026 21:48:30	3	55.49	1.26
41	14-10-2026 21:48:51	3	46.13	1.29
42	14-10-2026 21:49:11	3	47.22	1.29
43	14-10-2026 21:49:32	3	57.64	1.29
44	14-10-2026 21:49:52	3	48.73	1.29
45	14-10-2026 21:50:15	3	49.03	1.29
46	14-10-2026 21:50:36	3	50.8	1.29
47	14-10-2026 21:50:56	3	51.17	1.29
48	14-10-2026 21:51:18	3	59.91	1.29
49	14-10-2026 21:51:38	3	5.8	1.29
50	14-10-2026 21:52:00	2	0.01	1.29
51	14-10-2026 21:52:18	1	0.02	1.29
52	14-10-2026 21:52:35	1	0.04	1.29
53	14-10-2026 21:52:51	1	0.01	1.29
54	14-10-2026 21:53:07	1	0.08	1.29
55	14-10-2026 21:53:23	1	0.02	1.29
56	14-10-2026 21:53:40	1	0.01	1.29
57	14-10-2026 21:53:56	1	0.01	1.29
58	14-10-2026 21:54:12	1	0.02	1.29
59	14-10-2026 21:54:28	1	0.01	1.29
60	14-10-2026 21:54:45	1	0.01	1.29
61	14-10-2026 21:55:01	1	0.01	1.29
62	14-10-2026 21:55:17	1	0.02	1.29
63	14-10-2026 21:55:33	1	0.01	1.29
64	14-10-2026 21:55:50	1	0.04	1.29
65	14-10-2026 21:56:06	1	0.01	1.29
66	14-10-2026 21:56:22	1	0.07	1.29

	A	B	C
1	14-10-2026 21:46:41	2250	48
2	14-10-2026 21:46:46	2250	48
3	14-10-2026 21:46:51	2250	48
4	14-10-2026 21:46:56	2250	48
5	14-10-2026 21:47:01	2250	48
6	14-10-2026 21:47:06	2250	48
7	14-10-2026 21:47:11	2250	48
8	14-10-2026 21:47:16	2250	48
9	14-10-2026 21:47:21	2250	48
10	14-10-2026 21:47:27	2250	48
11	14-10-2026 21:47:27	2250	50
12	14-10-2026 21:47:32	5000	48
13	14-10-2026 21:47:37	5000	48
14	14-10-2026 21:47:42	5000	48
15	14-10-2026 21:47:47	5000	48
16	14-10-2026 21:47:52	5000	48
17	14-10-2026 21:47:57	5000	48
18	14-10-2026 21:48:02	5000	48
19	14-10-2026 21:48:07	5000	48
20	14-10-2026 21:48:12	5000	48
21	14-10-2026 21:48:17	5000	48
22	14-10-2026 21:48:22	5000	48
23	14-10-2026 21:48:27	5000	48
24	14-10-2026 21:48:32	5000	48
25	14-10-2026 21:48:37	5000	48
26	14-10-2026 21:48:42	5000	48
27	14-10-2026 21:48:47	5000	48
28	14-10-2026 21:48:52	5000	48
29	14-10-2026 21:48:57	5000	48
30	14-10-2026 21:49:02	5000	48

3. Live Plot Script (using Mathplotlib)



```python
import os
import time
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import pandas as pd

METRICS_FILE = "metrics.csv"
EXPERIMENT_FILE = "experiment_metrics.csv"
OUTPUT_IMAGE = "live_dashboard.png"
TEMP_IMAGE = "live_dashboard_tmp.png"
UPDATE_INTERVAL = 3

def parse_timestamp(df, col_name='timestamp'):
    df[col_name] = pd.to_datetime(df[col_name], format='%d-%m-%Y %H:%M:%S', errors='coerce')
    return df.dropna(subset=[col_name]).sort_values(col_name)

def update_dashboard():
    if not os.path.exists(METRICS_FILE) or os.stat(METRICS_FILE).st_size == 0:
        return

    try:
        df_sys = pd.read_csv(
            METRICS_FILE,
            header=None,
            names=['timestamp', 'containers', 'cpu', 'memory'],
            on_bad_lines='skip'
        )
        df_sys = parse_timestamp(df_sys, 'timestamp')
    except Exception:
        return

    if df_sys.empty:
        return

    df_exp = pd.DataFrame()
    if os.path.exists(EXPERIMENT_FILE) and os.stat(EXPERIMENT_FILE).st_size > 0:
        try:
            df_exp = pd.read_csv(
                EXPERIMENT_FILE,
                header=None,
                names=['timestamp', 'req_rate', 'resp_time'],
                on_bad_lines='skip'
            )
            df_exp = parse_timestamp(df_exp, 'timestamp')
        except Exception:
            df_exp = pd.DataFrame()

    fig, axes = plt.subplots(4, 1, figsize=(10, 11), sharex=True)
    plt.subplots_adjust(hspace=0.25)

    # Panel 1: Request Rate
    if not df_exp.empty and 'req_rate' in df_exp.columns:
        axes[0].plot(df_exp['timestamp'], df_exp['req_rate'], color='blue', marker='o',
        markersize=3, label='Request Rate (req/s)')
        axes[0].legend(loc='upper left', fontsize=8)
    # Panel 3: Response Time
    if not df_exp.empty and 'resp_time' in df_exp.columns:
        axes[2].plot(df_exp['timestamp'], df_exp['resp_time'], color='red',
        marker='o', markersize=3, label='Avg Response Time (ms)')
        axes[2].legend(loc='upper left', fontsize=8)
    axes[2].set_ylabel("Response Time (ms)", fontsize=9)
    axes[2].grid(True, linestyle='--', alpha=0.6)

    # Panel 4: Containers
    axes[3].plot(df_sys['timestamp'], df_sys['containers'], color='purple',
    marker='s', markersize=3, label='Running Containers')
    axes[3].set_ylabel("Containers", fontsize=9)
    axes[3].set_xlabel("Time", fontsize=9)
    axes[3].set_ylim(0.5, 4.5)
    axes[3].set_yticks([1, 2, 3, 4])
    axes[3].legend(loc='upper left', fontsize=8)
    axes[3].grid(True, linestyle='--', alpha=0.6)

    fig.autofmt_xdate()

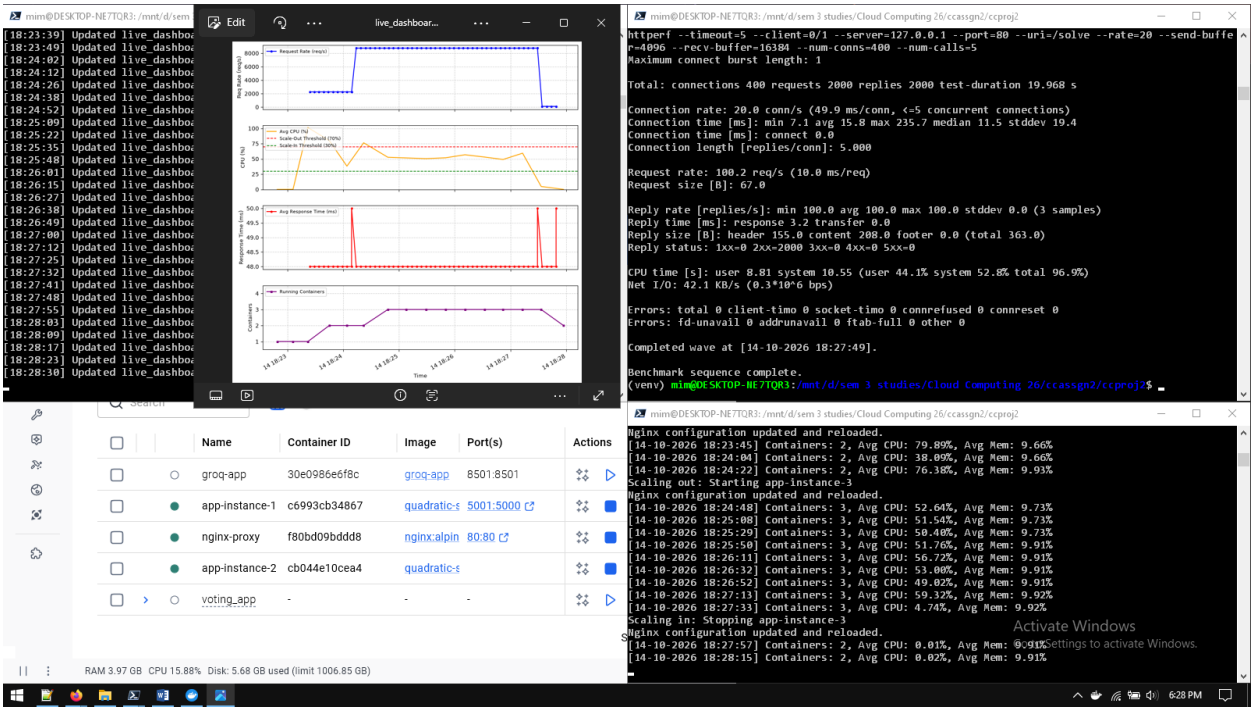
    try:
        fig.savefig(TEMP_IMAGE, bbox_inches='tight', dpi=100)
        plt.close(fig)
        os.replace(TEMP_IMAGE, OUTPUT_IMAGE)
        print(f"[{time.strftime('%H:%M:%S')}] Updated {OUTPUT_IMAGE}")
    except Exception as e:
        plt.close(fig)
        if os.path.exists(TEMP_IMAGE):
            try:
                os.remove(TEMP_IMAGE)
            except OSError:
                pass
        print(f"Skipped frame due to file access lock: {e}")

def main():
    print(f"Live plotter running. Updating '{OUTPUT_IMAGE}' every {
    UPDATE_INTERVAL}s...")
    while True:
        try:
            update_dashboard()
        except Exception as err:
            print(f"Error updating plot: {err}")
        time.sleep(UPDATE_INTERVAL)

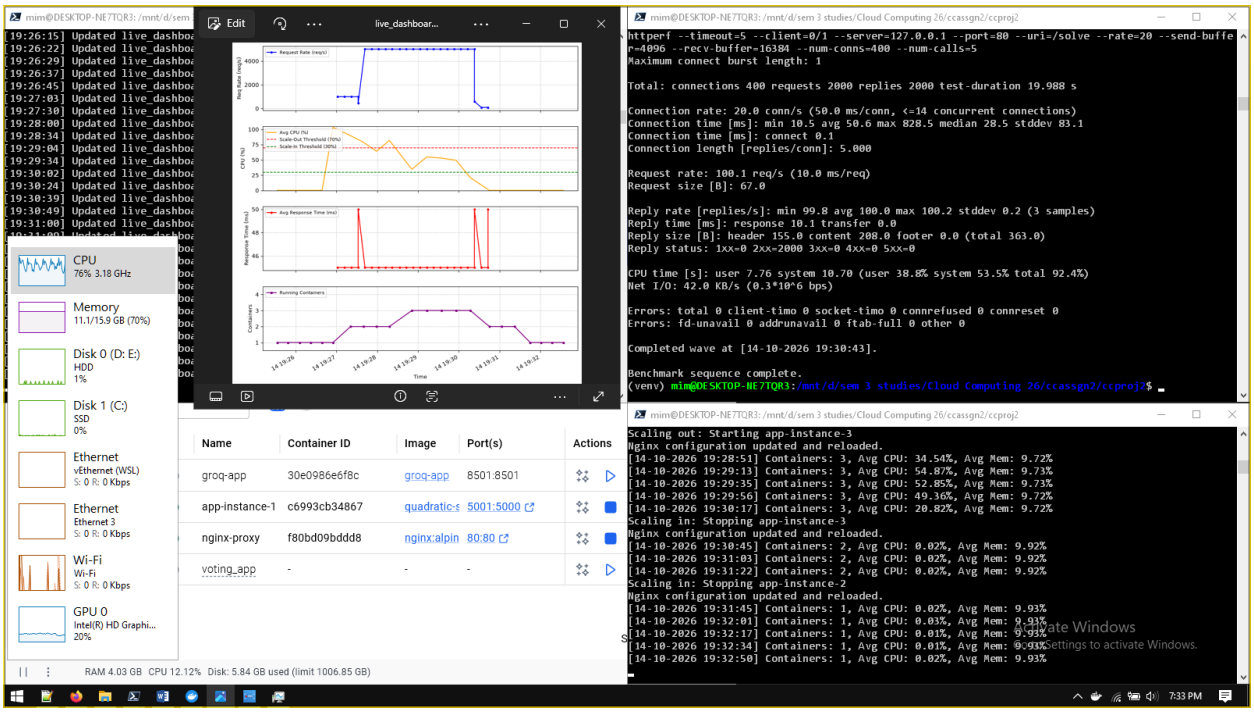
if __name__ == "__main__":
    main()
```

4. Test RUN with Synthetic load

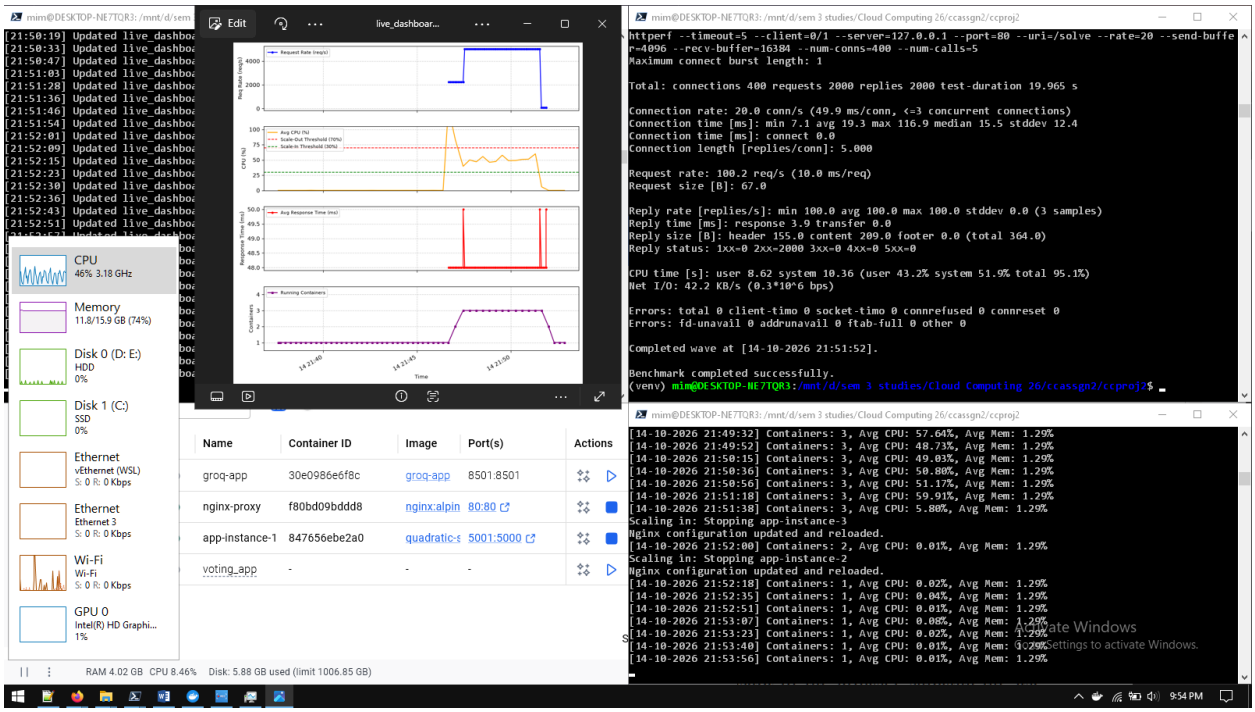
i) Test RUN 1:

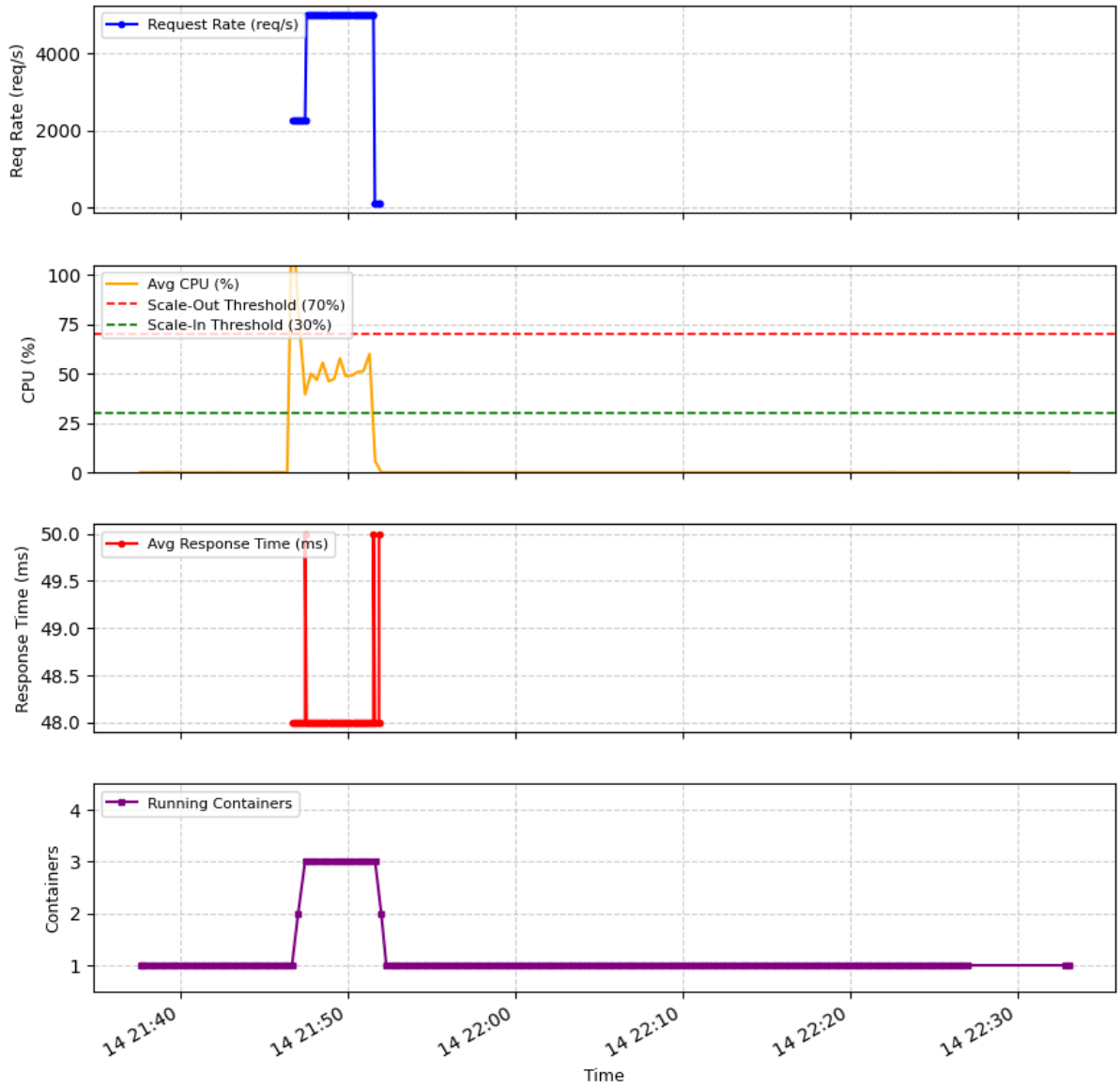


ii) Test Run 2:



iii) Test Run 3:




Request Rate (req/s)
Req Rate (req/s)
4000
2000
0
Avg CPU (%)
Scale-Out Threshold (70%)
Scale-In Threshold (30%)
CPU (%)
100
75
50
25
0
Avg Response Time (ms)
Response Time (ms)
50.0
49.5
49.0
48.5
48.0
Running Containers
Containers
4
3
2
1
14 21:40
14 21:50
14 22:00
14 22:10
14 22:20
14 22:30
Time

THE END